

KLE Society's
KLE Technological University



An Internship Project Report

On

SMART ATTENDANCE SYSTEM

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Engineering in

Computer Science And Engineering

Submitted By

Rohan S Dasogamath

01FE22BCS290

Under the guidance of

Prof. Vidya Handur

Associate Professor

COMPUTER SCIENCE AND ENGINEERING,
HUBBALLI-580 031 (India).

Academic year 2025-2026



**B. V. Bhoomaraddi College Campus, Vidyanagar, Hubballi - 580031.
Karnataka (India)**

COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that Internship Project entitled "**Smart Attendance System**" is a bonafide work carried out by the student **Rohan S Dasogamath** bearing USN **01FE22BCS290** in partial fulfillment of the completion of 8th semester B. E. course during the year 2025 – 26 at "**AarGees Business Solutions**". The Internship project report has been approved as it satisfies the academic requirement with respect to the project work prescribed for the above said course.

Name of the Guide
Prof. Vidya Handur

HoD
Dr. Vijayalakshmi M.

Name of the examiners

Signature with date

1 _____

2 _____



AarGees
Business Solutions

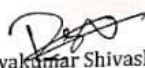


INTERNSHIP COMPLETION CERTIFICATE

This is to certify that Mr./Ms. **ROHAN .S. DASOGAMATH**, bearing **USN: 01FE22BCS290** studying in **KLE Technological University , Hubballi**, has successfully completed the internship training at **AarGees Business Solutions, Hubballi**, from **12/01/2026** to **30 /05/2026**.

During the internship period, the student worked on "**SMART ATTENDANCE**" and gained practical exposure in the field of **Software Development, Web Application Design, Database Management, and ERP Solutions**. The student's performance, dedication, sincerity, and involvement in the assigned tasks were found to be satisfactory.

We appreciate the efforts and commitment shown during the internship and wish him/her success in future academic and professional endeavors.


Vishwakumar Shivashimpiger
Internship Co-ordinator
AarGees Business Solutions


Guru Angadi
Director
AarGees Business Solutions



Date: 30/05/2026
Place: Hubballi

A Complete ERP Software Solutions for Educational Institutes

Hubballi : #01, 1st Floor, Lakeside Complex, Opp. Tolankere, Laxminagar,
Ravinagar Road, Hubballi - 580 030.

Bengaluru : #1AA-124-D, East of NGEF,
Kasturi Nagar, Bengaluru - 560 043

M : +91 98807 66013 / 14
Ph : 0836 4255123

Bank Details :

Name : AarGees Business Solutions Hubli

Bank Name : Union Bank of India, Vidyanagar, Hubli.

Current A/c No. : 510101000120080

IFSC Code : UBIN0905283

GST No. : 29AALFA9130C1ZK

www.aargees.com

sales@aargees.com, aargees2000@gmail.com

DECLARATION

I hereby declare that the Industry Project Report entitled “**Smart Attendance System**” is an authentic record of my own work as required by the industry, during the period from 12 Jan 2026 to 31 May 2026, for the award of the degree of B.E. under the guidance of Prof. Vidya Handur

(Signature of Student)

Rohan S Dasogamath

01FE22BCS290

DATE :

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of a number of individuals whose professional guidance and encouragement helped me in the successful completion of this report work.

I take this opportunity to express our sincere gratitude to Dr. Ashok Shettar, Pro Chancellor, Dr. Prakash Tewari, Vice Chancellor, and Dr. Basavaraj Anami, Registrar, KLE Technological University, Hubballi, for providing us with the opportunity and facilities to carry out this project successfully.

I would also like to thank Dr. Meena S. M., Dean, Faculty of Computer Science and Engineering, and Dr. Vijayalakshmi M., Head, Department of Computer Science and Engineering, for providing an excellent academic environment that nurtured our practical skills and contributed to the success of our project.

I sincerely thank our guide Prof. Vidya Handur for her guidance and wholehearted cooperation during the course of completion.

I sincerely thank my mentor Ms. Sindhu Basavraj, Software Engineer, AarGees Business Solution, for her support, inspiration, and wholehearted cooperation during the course of completion.

My gratitude will not be complete without thanking our beloved parents, our seniors, and our friends who have been a constant source of aspirations.

Swadeep Sawant
01FE22BCS308

ABSTRACT

Taking attendance in classrooms, especially in large batches, has always been a time consuming task for teachers. The traditional method of calling out names not only eats into lecture time but also does not stop students from marking attendance on behalf of their friends. This project is an attempt to solve this problem using face recognition technology.

The system captures a photograph of the classroom and automatically identifies which students are present. For this, the InsightFace buffalo_l model is used, which detects faces in the image and generates a 512-dimensional numerical representation called an embedding for each face. These embeddings are compared with the ones stored in the database at the time of student registration using cosine similarity. A threshold value of 0.35 is used to decide whether a face belongs to a registered student or not. To avoid counting the same student twice when multiple photos are uploaded, a duplicate check is also applied using a threshold of 0.70. The backend is built using Flask and exposes two endpoints, `/register` for enrolling students and `/recognize` for marking attendance. This Flask service is connected to an ASP.NET web application which handles the user interface, and a SQL Server database which stores student details, embeddings, timetable information and attendance records. The system supports four types of users namely Super Administrator, Administrator, Faculty and Student, each having their own set of permissions.

The final system is able to handle the complete attendance process starting from student registration to recognizing faces in classroom photos and saving the attendance records. It reduces the manual work involved and also removes the possibility of proxy attendance.

Keywords: Smart Attendance System, Face Recognition, InsightFace, Facial Embeddings, Cosine Similarity, Attendance Automation, Flask, ASP.NET, SQL Server.

CONTENTS

DECLARATION

ACKNOWLEDGEMENT i

ABSTRACT ii

CONTENTS iii

LIST OF TABLES v

LIST OF FIGURES vi

1	Introduction	1
1.1	Literature Survey	1
1.2	Motivation	2
1.3	Objectives	3
1.4	Problem Definition	4
2	Requirement Analysis	5
2.1	System Model	5
2.2	Functional Requirements	7
2.3	Non-Functional Requirements	9
2.4	Database Design	10
3	System Design	18
3.1	Architecture Design	18
3.2	Data Flow Diagram	24
4	Implementation	27
4.1	Module-wise Implementation Details	28
4.2	Face Recognition Model Implementation	29
4.3	Development Environment and Tools	31
5	TESTING	32
5.1	Unit Testing	32
5.1.1	Login Authentication Module	33
5.1.2	Student Registration Module	33
5.1.3	Faculty Registration Module	34
5.1.4	Face Recognition Module	35
5.1.5	Attendance Management Module	36

5.2	Integration Testing	37
5.3	Acceptance Testing	38
6	Results and Discussions	39
6.1	Achieved Functionalities	39
6.2	Performance Observations	41
6.3	Challenges Faced	41
6.4	Current Status	42
6.5	Discussion of Outcomes	42
7	Conclusion and Future Scope	44
7.1	Conclusion	44
7.2	Future Scope	45

LIST OF TABLES

5.1	Test Cases for Login Authentication Module	33
5.2	Test Cases for Student Registration Module	34
5.3	Test Cases for Faculty Registration Module	35
5.4	Test Cases for Face Recognition Module	36
5.5	Test Cases for Attendance Management Module	37

LIST OF FIGURES

2.1	Smart Attendance System - System Model (Layered Architecture)	5
3.1	Smart Attendance System - Architecture Design	18
3.2	Smart Attendance System - Architecture Design	21
3.3	Smart Attendance System Workflow	24
6.1	Student Login	39
6.2	Redirect to Dashboard	39
6.3	Stream Master Page	39
6.4	Stream List Page	39
6.5	Faculty Master Page	40
6.6	Student Bulk Import	40
6.7	Initial Face Recognition Result	40
6.8	Attendance Report	40
6.9	Face Recognition Accuracy	41

Chapter 1

Introduction

Educational institutions are a cornerstone of modern society, responsible for shaping skilled professionals and ensuring academic excellence. Maintaining accurate student attendance is a fundamental administrative task that directly impacts academic monitoring, discipline, and compliance with institutional regulations. However, traditional attendance systems, which rely on manual roll calls or paper-based registers, often lead to inefficiencies, errors, and time wastage. In large classrooms, particularly in medical colleges where class sizes can exceed 200 students, manual attendance consumes valuable lecture time and increases the risk of proxy attendance and inaccurate records.

In today's digital era, such manual processes act as significant bottlenecks, reducing productivity and affecting the overall learning experience. A smart and automated attendance system is no longer a luxury but a necessity for institutions aiming to improve efficiency and accuracy. This project focuses on developing a **Smart Attendance System using Face Recognition**, designed to automate and streamline the attendance process. The system captures classroom images, identifies students using trained facial recognition models, and records attendance automatically, ensuring speed, reliability, and transparency.

1.1 Literature Survey

Attendance systems have come a long way over the years. In the beginning, everything was done manually where teachers would call out names or pass around registers. Later, institutions started using digital methods like RFID cards and fingerprint scanners. These were definitely better than paper registers but they still had their own problems. Students had to physically interact with a device, queues would form, machines needed regular maintenance and in shared environments hygiene also became a concern.

As Artificial Intelligence, Machine Learning and Computer Vision started developing, researchers began exploring face recognition as a better alternative. These systems can identify a person just from a photograph without requiring any physical contact. Over time, face recognition itself has improved a lot. Earlier methods like Eigenfaces [1] worked on basic mathematical representations of faces. Later, deep learning based models like FaceNet [2] and ArcFace [3] were introduced which are far more accurate and can handle real world variations in lighting, pose and expression much better.

Face detection, which is the step before recognition, has also improved significantly. Models like MTCNN [4] can detect and align multiple faces in an image at the same time, which is useful when dealing with group photographs of an entire class.

Several studies have worked on camera based attendance systems that can process a classroom image and mark attendance for multiple students at once [5]. Such systems have shown that manual effort can be reduced considerably and proxy attendance can be eliminated in academic settings.

However, most of the existing systems have certain gaps. Many of them work only under controlled lighting conditions, do not scale well for larger batches and are not connected to any proper web platform for managing data. This project addresses these gaps by building a complete web based system where the face recognition backend, built using Flask, is integrated with an ASP.NET frontend and a SQL Server database for managing students, timetables and attendance records end to end..

1.2 Motivation

The motivation behind developing the Smart Attendance System using Face Recognition arises from the practical challenges observed in educational institutions, particularly in large classrooms such as medical colleges. Traditional attendance methods are highly time-consuming, as teachers are required to manually call out student names and record attendance during every lecture. In classes with a large number of students, this process significantly reduces effective teaching time and increases administrative workload.

Manual attendance systems are also prone to errors and misuse. Proxy attendance is a common issue, where students mark attendance on behalf of others, leading to inaccurate records. In addition, human errors such as missed entries or incorrect marking further reduce the reliability and authenticity of attendance data, making it difficult for institutions to maintain accurate records.

Another major challenge is the lack of a centralized system for managing attendance and student information. In many institutions, data is either maintained manually or stored across multiple disconnected systems. This results in data inconsistency, difficulty in accessing information, and inefficient workflows for both faculty and administration.

Handling attendance in large classrooms presents additional difficulties. Traditional biometric systems require students to stand in queues, which causes delays and congestion. Such systems are not suitable for environments where quick and efficient processing of attendance is required. In contrast, a camera-based system can capture multiple students simultaneously, making it more effective for large-scale implementation.

With the rapid advancement of Artificial Intelligence and Machine Learning, there is a growing need to automate repetitive and time-consuming tasks. AI-powered attendance systems significantly improve efficiency, accuracy, and reduce administrative workload. CNN-based attendance systems have demonstrated notable improvements in attendance automation [6].

Furthermore, an automated system improves transparency and student participation by allowing students to verify their attendance records and update their facial data when necessary. This ensures fairness and continuous improvement in system performance.

This project is therefore motivated by the need to develop a scalable, efficient, and intelligent attendance management system that reduces manual effort, improves accuracy, saves time, and enhances the overall academic workflow.

1.3 Objectives

The primary objective of this project is to design, develop and implement a smart attendance system using face recognition that reduces manual effort and improves accuracy in tracking student attendance. The specific objectives are as follows:

1. **Web-Based Accessibility:** Build a web application that can be accessed from desktops, laptops and mobile devices so that faculty and students are not restricted to a single machine.
2. **Automated Attendance Marking:** Automatically detect and identify student faces from uploaded classroom photographs and mark attendance without any manual input from the faculty.
3. **Student Enrollment and Data Management:** Allow students to register their facial data through an image upload portal and maintain their profiles, embeddings and attendance history in a centralised database.
4. **Face Recognition Using Pretrained Model:** Use the InsightFace buffalo_l pretrained model to extract 512-dimensional facial embeddings and perform accurate recognition using cosine similarity matching.
5. **Multi-Face Processing:** Handle classroom photographs containing multiple students simultaneously and correctly identify each one in a single recognition request.
6. **System Integration:** Connect the Python based Flask recognition service with the ASP.NET frontend and SQL Server database so that all components work together as one complete system.
7. **Accuracy and Reliability:** Reduce errors in attendance marking by applying a similarity threshold for recognition and a duplicate suppression mechanism to avoid recording the same student more than once.
8. **User-Friendly Interface:** Provide separate dashboards for Super Administrator, Administrator, Faculty and Student with only the features relevant to each role.

9. **Scalability:** Design the system in a modular way so that it can be extended to handle larger student batches, multiple classrooms and integration with institutional ERP systems in the future.

1.4 Problem Definition

The core problem addressed in this project is the inefficiency, inaccuracy, and time consumption associated with traditional attendance systems in educational institutions. Manual attendance methods are prone to human errors, proxy attendance, and delays, especially in large classrooms with hundreds of students.

Existing digital solutions such as biometric systems often require physical interaction, involve high costs, and fail to scale efficiently for large groups. Additionally, many systems lack real-time processing and centralized management, making it difficult to maintain accurate and accessible attendance records.

There is a need for an intelligent, automated system that can recognize students in real-time without manual intervention. The system must handle multiple faces simultaneously, work under varying conditions, and integrate seamlessly with a web-based platform.

This project aims to address these challenges by developing a facial recognition-based attendance system that automates attendance marking, improves accuracy, reduces time consumption, and enhances overall efficiency in academic institutions.

Chapter 2

Requirement Analysis

Requirement analysis is a crucial phase in software development, aimed at understanding and documenting the needs of stakeholders and defining the functionalities the system must provide. For the Smart Attendance System, this phase involved identifying challenges in traditional attendance methods and gathering requirements from administrators, faculty, and students. The requirements are categorized into system model, functional requirements, non-functional requirements, and database requirements to ensure a structured and efficient system design.

2.1 System Model

The Smart Attendance System is built as a multi-tier web based application. It brings together a face recognition backend and a web interface so that attendance can be marked from classroom photographs without any manual effort. The system has four types of users: Super Administrator, Administrator, Faculty and Student.

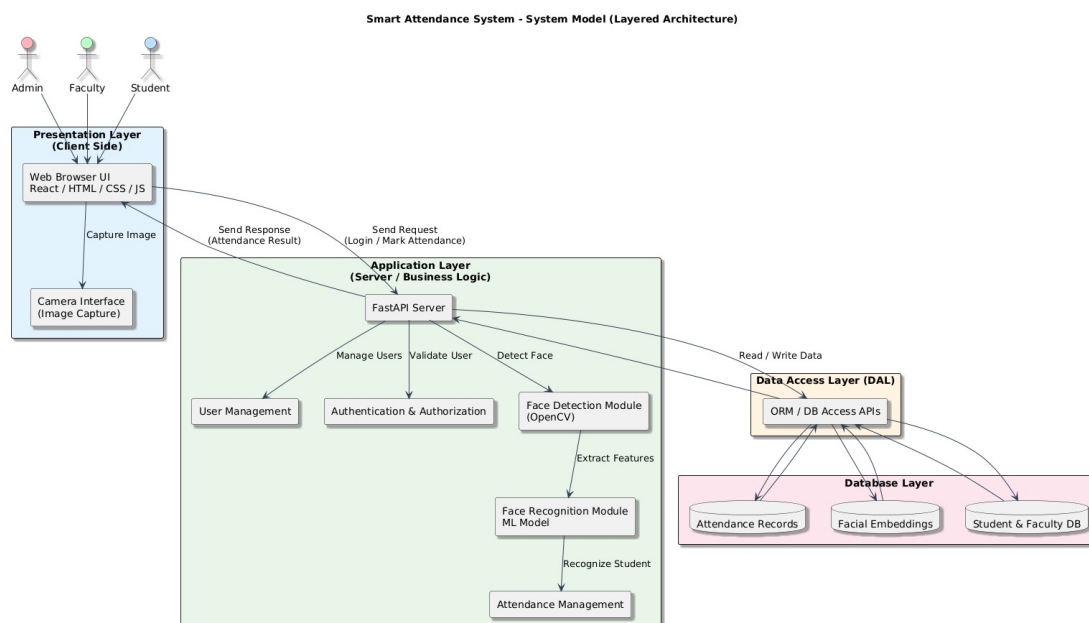


Figure 2.1: Smart Attendance System - System Model (Layered Architecture)

The system is organised into four layers as described below.

Presentation Layer (Client-Side)

- Accessible from desktop, tablet and mobile browsers.
- Built using ASP.NET Web Forms with the BootstrapDash template for responsive design.

- Provides login, role-based dashboards, student management forms and attendance pages.
- Allows faculty to upload classroom photographs for attendance marking.
- Displays recognition results and attendance records to the respective users.
- Communicates with the Application Layer by sending HTTP requests and displaying the responses received.

Application Layer (Server-Side / Business Logic Layer)

- The core of the system responsible for handling all business logic.
- Built using ASP.NET with C# for the main web application.
- A separate Python based Flask service handles face detection and recognition using the InsightFace buffalo_l model.
- The ASP.NET application communicates with the Flask service by sending uploaded images and receiving recognition results.
- Handles user authentication using session and cookie based mechanisms.
- Implements role-based access control for all four user types.
- Manages student registration, faculty assignment, timetable creation and attendance processing.

Data Access Layer (DAL)

- Acts as a bridge between the Application Layer and the Database Layer.
- Encapsulates all database operations so that the business logic does not directly interact with the database.
- Ensures data integrity and consistency across all modules.
- Uses direct database query interfaces to communicate with SQL Server.

Database Layer

- Responsible for storing all system data in a structured and persistent manner.
- Built on Microsoft SQL Server managed through SQL Server Management Studio (SSMS).
- Stores student profiles, faculty details, facial embeddings, timetable schedules and attendance records.
- Maintains relationships between entities using primary and foreign keys to ensure referential integrity.

Flow of Information

- A user such as a faculty member logs in and uploads a classroom photograph through the web interface.
- The ASP.NET application forwards the image to the Flask recognition service.
- The Flask service detects all faces in the image, extracts embeddings and compares them against stored student embeddings in the database.
- Recognition results are sent back to the ASP.NET application.
- The application marks attendance for recognised students and saves the records in SQL Server.
- The updated attendance data is displayed on the faculty and student dashboards.

This layered design keeps each part of the system independent so that changes in one layer do not affect the others, making the system easier to maintain and extend in the future.

2.2 Functional Requirements

Functional requirements describe what the system is supposed to do from the user's point of view. The key functional requirements for this system are listed below.

I. User Management

- User Registration: Administrators can add new faculty and student accounts to the system.
- Login and Logout: All users authenticate using a username and password. Sessions are maintained using cookies.
- Role-Based Access Control: Four roles are supported, namely Super Administrator, Administrator, Faculty and Student. Each role has access only to the features relevant to it.
- Profile Management: Users can view and update their own profile details including their photograph.
- Password Management: Users can change their password from within the application.

II. Student Management

- Add, Edit and Delete Students: Administrators can manage student records individually or through bulk import using an Excel file.

- **Student Information:** The system stores details such as name, student ID, division, course, stream and contact information.
- **Facial Data Enrollment:** Students upload multiple photographs of themselves which are used to generate and store facial embeddings for recognition.
- **Student Search:** Students can be searched by name, ID or other available details.

III. Face Recognition

- **Face Detection:** The system detects all faces present in an uploaded classroom photograph.
- **Face Recognition:** Detected faces are matched against stored embeddings using cosine similarity to identify the corresponding students.
- **Multiple Face Handling:** The system processes all faces found in a single photograph in one request, making it suitable for full classroom images.
- **Duplicate Prevention:** A duplicate suppression check ensures the same student is not counted more than once even if multiple photographs are uploaded for the same session.

IV. Attendance Management

- **Automatic Attendance Marking:** Once students are recognised, their attendance is marked as present without any manual input.
- **Record Storage:** Attendance entries are saved with date, time, subject and division information.
- **Attendance Viewing:** Faculty can view class-wise attendance and students can view their own attendance records from their dashboards.
- **Filtering:** Attendance records can be filtered by date, student, subject or division.
- **Report Generation:** Attendance reports can be generated and downloaded in CSV format for further use.

V. System Operations

- **Processing Speed:** The recognition and attendance marking process completes within a few seconds for a standard classroom photograph.
- **Data Synchronisation:** All changes made through the application are immediately reflected in the SQL Server database.
- **Error Handling:** The system handles cases where no face is detected, a student is not registered or a database error occurs, and returns a clear message to the user.

2.3 Non-Functional Requirements

Non-functional requirements (NFRs) specify criteria that can be used to judge the operation of a system, rather than specific behaviors. They are crucial for the overall quality and success of the Smart Attendance System.

I. Performance:

- **Response Time:** All critical operations (e.g., face detection, face recognition, attendance marking) shall complete within 3 seconds under normal load.
- **Concurrency:** The system shall support at least 50 concurrent users (students and faculty) without significant degradation in performance.
- **Load Handling:** The system shall be able to handle large classroom attendance efficiently in a single session.
- **Data Retrieval:** Search queries for attendance records or student data shall return results within 2 seconds.

II. Security:

- **Authentication:** All users must be authenticated via unique usernames and strong passwords. Implement multi-factor authentication (MFA) as a future enhancement.
- **Authorization:** Implement role-based access control (RBAC) to ensure users can only access functionalities and data relevant to their assigned roles.
- **Data Encryption:** Sensitive data (e.g., student personal details and facial data) shall be encrypted during transmission and at rest.
- **Input Validation:** All user inputs shall be validated to prevent common system vulnerabilities.
- **Audit Trails:** The system shall maintain logs of critical actions performed by users (e.g., attendance updates, data modifications).

III. Usability:

- **User Interface (UI):** The UI shall be consistent and easy to navigate, minimizing the learning curve for new users.
- **Error Handling:** The system shall provide clear and user-friendly error messages, guiding users on how to resolve issues.

- **Accessibility:** The UI should adhere to basic accessibility guidelines to accommodate users with different needs.

IV. Reliability:

- **Availability:** The system shall be available at all times, excluding scheduled maintenance.
- **Data Consistency:** The system shall ensure data consistency across all modules, preventing discrepancies in attendance records.
- **Backup and Recovery:** Implement a robust data backup and recovery plan to prevent data loss in case of failures.

V. Scalability:

- **Horizontal Scalability:** The architecture should allow for easy addition of more users and data handling capacity.
- **Vertical Scalability:** The system should be able to utilize more powerful hardware if needed.
- **Database Scalability:** The database design should support growth in attendance data without performance degradation.

VI. Compatibility:

- **Browser Compatibility:** The web application shall be compatible with major modern web browsers (Chrome, Firefox, Edge, Safari).

VII. Portability:

- The system should be designed to be deployable on various cloud platforms or local servers with minimal configuration changes.

2.4 Database Design

The database is the backbone of the Smart Attendance System. It stores and manages all information related to institutions, users, students, faculty, academic structure, timetables, facial embeddings and attendance records. Microsoft SQL Server is used as the database management system, accessed through SQL Server Management Studio (SSMS). The database is named `students_attendance`.

The schema consists of fifteen tables whose structure is described below.

I. Society

Stores the top level entity. Each society can manage multiple institutes.

- SC — Primary Key, Auto-increment
- Name — NVARCHAR, Unique

II. Institute

Stores institute details. Each institute belongs to one society.

- IC — Primary Key, Auto-increment
- InstituteName — NVARCHAR
- Phone, Email — NVARCHAR
- State, District, Location, Pincode — NVARCHAR
- ContactPersonName, ContactPersonPhone — NVARCHAR
- GSTNumber, PANCard — NVARCHAR
- CreatedOn — DATETIME, defaults to current timestamp
- SC — Foreign Key to Society

III. Usermaster

Stores admin level user accounts for each institute.

- id — Primary Key, Auto-increment
- UserName, PasswordHash — NVARCHAR
- ProfileImage — VARBINARY
- FirstName, LastName — NVARCHAR
- MobileNo, Email — NVARCHAR
- IC — Foreign Key to Institute
- SC — Foreign Key to Society
- user_type, user_id — NVARCHAR
- isActive — BIT
- IsOTPEntered — BIT

IV. SuperAdmin

Stores super administrator accounts who manage the overall system.

- `id` — Primary Key, Auto-increment
- `UserName, PasswordHash` — NVARCHAR
- `ProfileImage` — VARBINARY
- `FirstName, LastName` — NVARCHAR
- `MobileNo, Email` — NVARCHAR
- `IC` — Foreign Key to Institute
- `SC` — Foreign Key to Society
- `user_type, user_id` — NVARCHAR
- `isActive` — BIT
- `IsOTPEntabled` — BIT

V. AcademicYear

Stores academic year records. Only one year is active at a time.

- `ID` — Primary Key, Auto-increment
- `YearCode` — NVARCHAR, Unique (e.g. 2024-25)
- `FromDate, ToDate` — DATE
- `YearDescription` — NVARCHAR
- `OldYear` — NVARCHAR
- `IsActive` — BIT

VI. StreamMaster

Stores academic streams such as B.E., M.Tech, MBBS.

- `StreamId` — Primary Key, Auto-increment
- `StreamCode, StreamName` — VARCHAR
- `AcademicYear` — VARCHAR
- `DurationYears` — INT

- IC, SC — Foreign Keys
- IsActive — BIT
- CreatedDate — DATETIME

VII. CourseMaster

Stores courses that belong to a stream, for example first year or second year of a programme.

- CourseId — Primary Key, Auto-increment
- CourseCode, CourseName — VARCHAR
- StreamId — Foreign Key to StreamMaster
- AcademicYear — VARCHAR
- IC, SC — Foreign Keys
- IsActive — BIT
- CreatedDate — DATETIME

VIII. SubjectMaster

Stores subjects taught in a particular course and stream.

- SubjectId — Primary Key, Auto-increment
- SubjectCode, SubjectName — VARCHAR
- StreamId — Foreign Key to StreamMaster
- CourseId — Foreign Key to CourseMaster
- AcademicYear — VARCHAR
- IC, SC — Foreign Keys
- IsActive — BIT
- CreatedDate — DATETIME

IX. DivisionMaster

Stores divisions or sections within a course, for example Division A or Division B.

- DivisionId — Primary Key, Auto-increment
- DivisionCode, DivisionName — VARCHAR
- StreamId — Foreign Key to StreamMaster
- CourseId — Foreign Key to CourseMaster
- AcademicYear — VARCHAR
- IC, SC — Foreign Keys
- IsActive — BIT
- CreatedDate — DATETIME

X. SlotMaster

Stores the time slots defined for a division such as morning or afternoon lecture periods.

- SlotId — Primary Key, Auto-increment
- SC, IC — Foreign Keys
- AcademicYear — VARCHAR
- StreamId, CourseId, DivisionId — Foreign Keys
- DayOfWeek — VARCHAR
- SlotPriority — INT
- StartTime, EndTime — TIME
- CreatedBy — INT
- CreatedDate — DATETIME

XI. FacultyMaster

Stores faculty member details including their login credentials and subject assignment.

- Faculty_id — Primary Key, Auto-increment
- Emp_id — NVARCHAR
- user_type — NVARCHAR

- UserName, PasswordHash — NVARCHAR
- FirstName, LastName — NVARCHAR
- MobileNo, Email — NVARCHAR
- ProfileImage — VARBINARY
- StreamId — Foreign Key to StreamMaster
- CourseId — Foreign Key to CourseMaster
- SubjectId — Foreign Key to SubjectMaster
- IC, SC — Foreign Keys
- IsActive — BIT
- CreatedDate — DATETIME

XII. StudentRegister

Stores complete student information including personal details, academic assignment and login credentials.

- StudentId — Primary Key, Auto-increment
- StudentIdNo — NVARCHAR
- FirstName, MiddleName, LastName — NVARCHAR
- Gender, DOB, Category, BloodGroup — NVARCHAR / DATE
- FatherName, MotherName — NVARCHAR
- MobileNo, ParentMobileNo, Email — NVARCHAR
- Address — NVARCHAR(MAX)
- AcademicYear — NVARCHAR
- StreamId, CourseId, DivisionId — Foreign Keys
- AdmissionNo, AdmissionDate — NVARCHAR / DATE
- Photo — VARBINARY
- LoginID, PasswordHash — NVARCHAR
- user_type — CHAR, default value S for Student
- IC, SC — Foreign Keys
- isActive — INT

XIII. TimeTable

Links a faculty member to a subject, division and slot for a given academic year, forming the complete lecture schedule.

- TimeTableId — Primary Key, Auto-increment
- SlotId — Foreign Key to SlotMaster
- SC, IC — Foreign Keys
- AcademicYear — VARCHAR
- StreamId, CourseId, DivisionId — Foreign Keys
- SubjectId — Foreign Key to SubjectMaster
- FacultyId — Foreign Key to FacultyMaster
- SlotPriority — INT
- CreatedBy — INT
- CreatedDate — DATETIME

XIV. StudentEmbeddings

Stores the facial embeddings generated from student photographs during the enrollment phase. Each student can have multiple embeddings to improve recognition accuracy.

- Id — Primary Key, Auto-increment
- StudentId — Foreign Key to StudentRegister
- Embedding — NVARCHAR(MAX), stores the 512-dimensional feature vector in JSON format
- CreatedAt — DATETIME, defaults to current timestamp

XV. AttendanceRecords

Stores the final attendance outcome for each student per session. Both present and absent entries are recorded so that a complete attendance history is maintained.

- AttendanceId — Primary Key, Auto-increment
- SessionId — NVARCHAR, uniquely identifies one attendance session (e.g. 20250521_143022)
- SlotId, TimeTableId — Foreign Keys

- FacultyId — Foreign Key to FacultyMaster
- SubjectId, SubjectName — Foreign Key and NVARCHAR
- StreamId, CourseId, DivisionId — Foreign Keys
- AcademicYear — NVARCHAR
- StudentId — Foreign Key to StudentRegister
- StudentIdNo, StudentName — NVARCHAR
- IC, SC — Foreign Keys
- AttendanceDate — DATE
- AttendanceTime — TIME
- AttendanceStatus — NVARCHAR, either Present or Absent
- RecognitionScore — FLOAT, cosine similarity score for present entries, NULL for absent
- CreatedAt — DATETIME, defaults to current timestamp

Relationships

- One Society can have multiple Institutes.
- One Institute can have multiple Users, Super Admins, Students and Faculty members.
- One AcademicYear is linked to multiple Streams, Courses, Subjects and Divisions.
- One Stream can have multiple Courses and each Course can have multiple Subjects and Divisions.
- One Student belongs to one Division and one Course within an academic year.
- One Faculty member is assigned to one Subject within a Stream and Course.
- SlotMaster defines the time period of a lecture for a given Division.
- TimeTable connects Faculty, Subject, Division and Slot to form the complete lecture schedule.
- StudentEmbeddings stores one or more facial embeddings per student, which are used during the recognition process.
- AttendanceRecords stores one entry per student per session, with the status set to either Present or Absent based on the recognition result.

Chapter 3

System Design

This chapter covers how the Smart Attendance System is structured internally. It describes the overall architecture, how different components talk to each other, and how data moves through the system when attendance is being marked. The design decisions taken here are aimed at keeping the system simple to maintain while also making sure it can handle real classroom scenarios without breaking down.

3.1 Architecture Design

The system is built on a three tier architecture where the user interface, the business logic and the data storage are kept separate from each other. In addition to these layers, a dedicated AI based face recognition service is integrated to automate student identification and attendance marking. This separation of responsibilities improves maintainability, scalability and allows individual components to be upgraded independently without affecting the rest of the system.

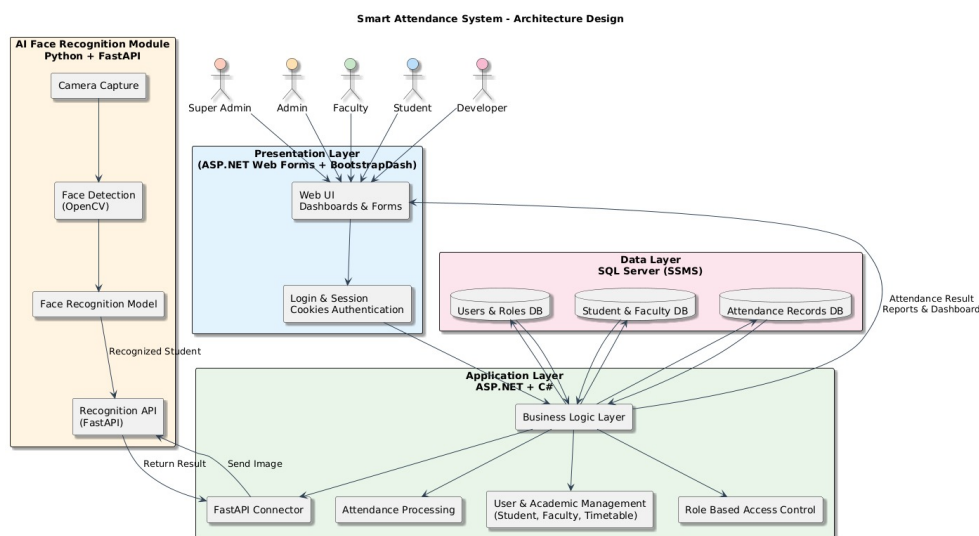


Figure 3.1: Smart Attendance System - Architecture Design

Presentation Layer

- Built using ASP.NET Web Forms with the BootstrapDash template.
- Supports desktop, tablet and mobile browsers through a responsive user interface.
- Provides separate dashboards for Developer, Super Administrator, Administrator, Faculty and Student.

- Allows faculty members to upload classroom photographs for attendance marking.
- Displays attendance reports, student records, recognition results and system notifications.

Application Layer

- Implemented using ASP.NET and C#.
- Handles business logic, authentication and authorization.
- Enforces role based access control for all users.
- Manages academic data including streams, courses, subjects, divisions and timetables.
- Processes student registration, faculty management and attendance operations.
- Acts as an intermediary between the web interface, database and AI recognition service.
- Sends classroom photographs to the Flask service and receives recognition results.
- Stores attendance information returned by the AI module into SQL Server.

AI Module

The AI Module forms the intelligence layer of the Smart Attendance System and is responsible for face detection, feature extraction, student identification and attendance recognition. The module is implemented as a standalone Python service using the Flask framework and communicates with the ASP.NET application through REST API requests.

The system uses the InsightFace `buffalo_l` pretrained model for face analysis. This model is capable of detecting multiple faces in a classroom photograph and generating robust facial representations even under variations in lighting, pose and facial expressions.

The face recognition workflow consists of the following stages:

1. Classroom photographs uploaded by faculty are received by the Flask service.
2. The InsightFace `buffalo_l` model detects all faces present in the image and generates bounding boxes.
3. For each detected face, a 512-dimensional facial embedding is extracted. This embedding acts as a numerical representation of a person's facial characteristics.
4. During student enrollment, multiple photographs are collected from each student and corresponding embeddings are stored in the `StudentEmbeddings` table.
5. The generated embeddings are compared against stored student embeddings using cosine similarity.

6. A recognition threshold of 0.35 is used to determine whether a detected face belongs to a registered student.
7. To prevent duplicate attendance entries when multiple photographs are uploaded for the same session, a duplicate filtering mechanism is applied using a similarity threshold of 0.70.
8. The service generates a response containing recognised students, similarity scores, unknown faces and an annotated classroom image.
9. The recognition results are returned to the ASP.NET application for attendance processing.

The modular design of the AI service allows future enhancements such as GPU acceleration, liveness detection, real-time video processing and improved face recognition models without requiring modifications to the main web application.

Data Layer

- Microsoft SQL Server managed through SQL Server Management Studio (SSMS) serves as the database platform.
- Stores institutional data, user accounts, academic information, timetables and attendance records.
- Maintains facial embeddings generated during student enrollment.
- Uses primary key and foreign key relationships to ensure referential integrity.
- Provides persistent and centralized storage for all application data.

Data Flow

- Faculty members log in and upload one or more classroom photographs through the web interface.
- The ASP.NET application validates the request and forwards the images to the Flask recognition service.
- The AI module detects faces present in the uploaded photographs.
- A 512-dimensional embedding is generated for each detected face.
- The generated embeddings are compared against stored student embeddings using cosine similarity matching.
- Students whose similarity scores satisfy the recognition threshold are identified as recognised students.

- Duplicate detections are removed using the duplicate suppression mechanism.
- The Flask service returns recognised student information, similarity scores and an annotated classroom image.
- The ASP.NET application automatically marks recognised students as Present and remaining students as Absent.
- Attendance records are stored in SQL Server and immediately become visible on faculty and student dashboards.

Benefits of This Design

- Each layer can be developed and maintained independently.
- The AI recognition service can be upgraded without affecting the ASP.NET application.
- Role based access control improves security and data privacy.
- Centralized database storage ensures consistency of attendance records.
- The architecture supports future expansion to larger student batches and multiple institutions.
- New AI features such as liveness detection and live camera integration can be added without redesigning the entire system.

The system is built on a three tier architecture where the user interface, the business logic and the data storage are kept separate from each other. On top of this, a Python based face recognition service is connected to handle all the AI related work. Keeping these parts separate means that if something needs to be changed in one part, the other parts do not get affected.

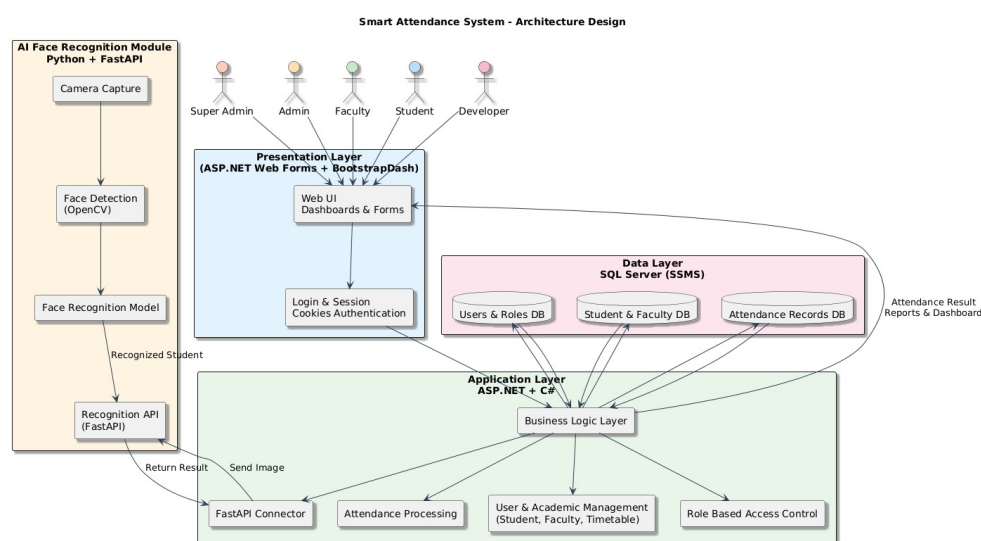


Figure 3.2: Smart Attendance System - Architecture Design

Presentation Layer

- Built using ASP.NET Web Forms with the BootstrapDash template.
- Works on desktops, tablets and mobile browsers without any separate mobile application.
- Provides separate dashboards for Developer, Super Administrator, Administrator, Faculty and Student based on their role.
- Faculty can upload classroom photographs for attendance marking from this layer.
- Attendance records, student details and generated reports are all displayed here.

Application Layer

- Written in C# using ASP.NET, this layer handles all the business logic of the system.
- Takes care of user authentication using session and cookie based mechanisms and enforces role based access control.
- Manages student registration, faculty assignment, timetable creation and attendance processing.
- When attendance needs to be marked, this layer sends the uploaded classroom photograph to the Flask recognition service and waits for the result.
- Once the recognition result is received, it saves the attendance entries into the SQL Server database.

AI Module

- A standalone Python service built using Flask.
- Receives classroom photographs sent by the ASP.NET application through HTTP requests.
- Uses the InsightFace buffalo_l pretrained model to detect faces in the photograph and extract 512-dimensional embeddings for each detected face.
- Compares these embeddings with the ones stored in the database using cosine similarity to identify which students are present.
- Sends back the list of recognised students along with their similarity scores and an annotated version of the photograph to the ASP.NET application.

Data Layer

- Microsoft SQL Server managed through SSMS serves as the database.
- Stores all data related to institutions, users, students, faculty, timetables, facial embeddings and attendance records.
- Foreign key relationships between tables make sure the data stays consistent across all modules.

Data Flow

- Faculty logs in and uploads one or more classroom photographs through the web interface.
- The ASP.NET application receives the photographs and forwards them to the Flask service.
- The Flask service detects all faces, extracts embeddings and matches them against stored student embeddings.
- The recognition result is returned to the ASP.NET application.
- Attendance is marked as Present or Absent for each student and saved in the database.
- The updated attendance is immediately visible on the faculty and student dashboards.

Benefits of This Design

- Each layer can be worked on independently without disturbing the rest of the system.
- The Flask recognition service can be updated or replaced without changing the ASP.NET application.
- Role based access ensures that each user only sees what they are supposed to see.
- The system can be extended to support multiple institutions and larger student batches without redesigning from scratch.

3.2 Data Flow Diagram

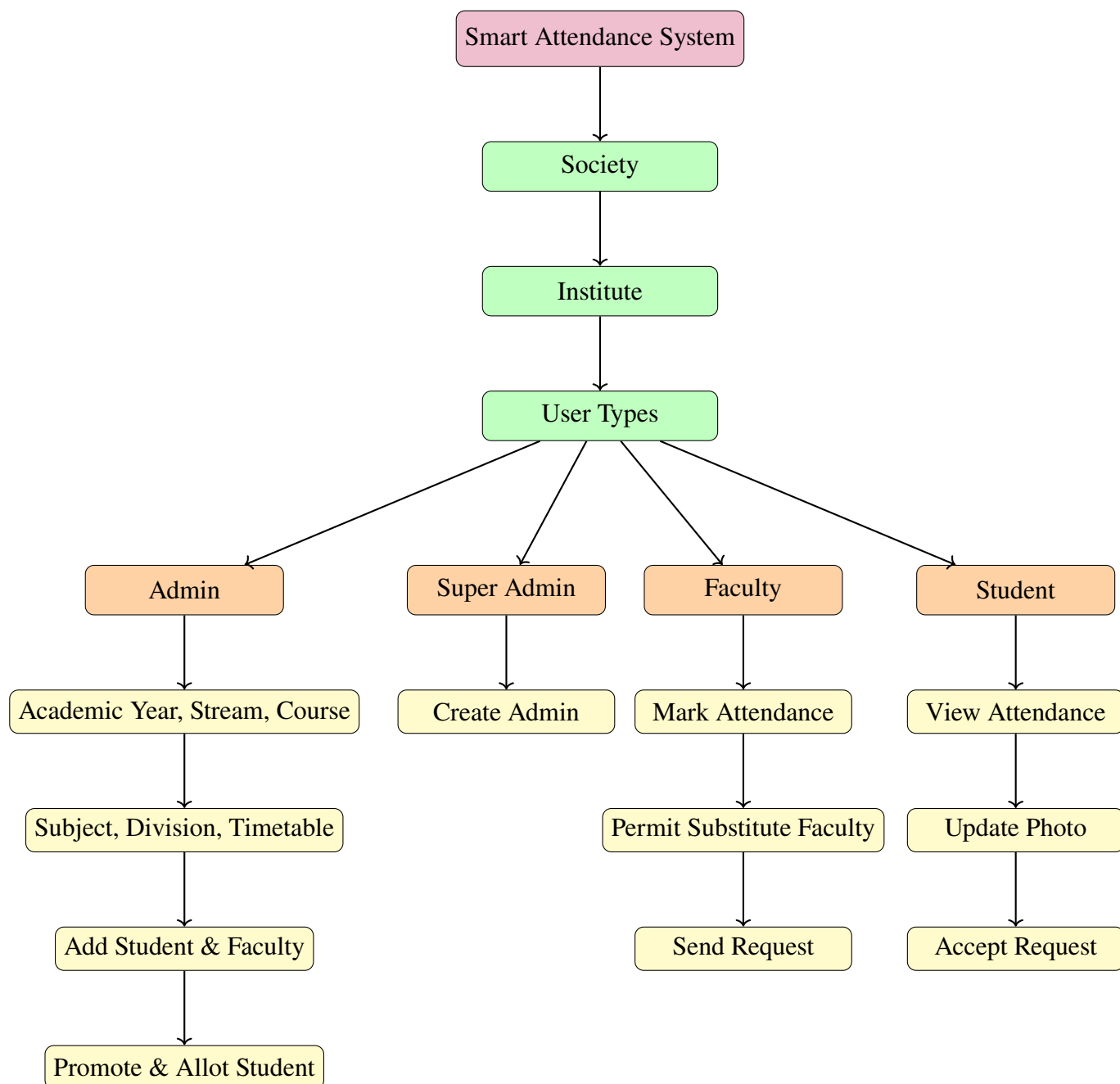


Figure 3.3: Smart Attendance System Workflow

The data flow diagram shows how information enters and leaves the system through different users. The Smart Attendance System sits at the centre and interacts with five external entities: Developer, Super Administrator, Administrator, Faculty and Student. Each entity has specific inputs they provide to the system and outputs they receive in return.

I. Central Process

- The Smart Attendance System acts as the main processing unit.

- It handles student registration, facial enrollment, timetable management, attendance marking and report generation.
- All external entities interact with it based on their assigned role.

II. External Entities and Their Interactions

a. Developer

Input to System:

- Society and institute setup details to configure the system for a new organisation.
- Super Administrator account creation to hand over control to the institution.

Output from System:

- Confirmation that the society, institute and super admin have been created successfully.

b. Super Administrator

Input to System:

- Administrator account creation details.
- User role and permission management data.

Output from System:

- Newly created administrator account details.
- Role and access configuration confirmation.

c. Administrator

Input to System:

- Academic year, stream, course, subject and division setup data.
- Student and faculty registration data, either individually or through bulk Excel import.
- Timetable creation data linking faculty, subjects and time slots.
- Student promotion and division allocation details.

Output from System:

- Saved academic structure records.
- Registered student and faculty profiles.
- Generated timetable schedules.
- Attendance and academic reports.

d. Faculty**Input to System:**

- Classroom photographs uploaded for face recognition based attendance.
- Substitute permission requests for assigning an alternate faculty member to a slot.
- Approval or rejection decisions on student attendance correction requests.

Output from System:

- Attendance records showing which students were marked present or absent along with the annotated classroom photograph.
- Status updates on substitute and correction requests.

e. Student**Input to System:**

- Facial photographs uploaded during the enrollment phase for generating recognition embeddings.
- Attendance correction requests submitted when attendance seems incorrect.

Output from System:

- Personal attendance dashboard showing subject wise attendance percentage and records.
- Notifications on the status of submitted correction requests.

f. Face Recognition Module**Input from System:**

- Classroom photographs forwarded by the ASP.NET application for processing.[7]

Output to System:

- List of recognised students with their identity details and cosine similarity scores.
- List of unrecognised faces detected in the photograph.
- Annotated group photograph with bounding boxes drawn around each detected face.

Chapter 4

Implementation

This chapter explains how the Smart Attendance System was actually built. It covers the technologies used, how each module works and how the face recognition part was put together and connected to the rest of the system.

The system is made up of four main parts. The frontend which the users interact with, the backend which handles all the logic, the database which stores everything and the face recognition service which does the actual identification work.

1. Frontend

i. ASP.NET Web Forms

The entire user interface is built using ASP.NET Web Forms. It follows an event driven model which made it straightforward to build forms, dashboards and data tables for different user roles.

ii. BootstrapDash Template

BootstrapDash is used as the base UI template. It provides a responsive layout that adjusts properly on desktops, tablets and mobile screens without any extra effort.

iii. HTML5 and CSS3

Used for the structure and styling of all web pages along with the BootstrapDash template.

iv. JavaScript

Used for client side validation, dynamic behaviour and handling user interactions on forms and dashboards.

2. Backend

ASP.NET with C#

- Handles all server side processing and business logic.
- Authentication is done using session and cookie based mechanisms.
- Role based access control is applied so each user only accesses what their role allows.
- Connects the frontend with the SQL Server database and with the Flask face recognition service.

3. Database

SQL Server managed through SSMS

- All system data including users, students, faculty, timetables, embeddings and attendance records are stored here.
- Table relationships are maintained using primary and foreign keys.
- Data integrity is enforced at the database level through constraints.

4. AI Module

Python Face Recognition Service using Flask

- Faculty uploads classroom photographs through the web interface.
- The Flask service receives these photographs, detects faces and identifies students using the InsightFace buffalo_l pretrained model.
- Recognition results are sent back to the ASP.NET application which then saves the attendance.

4.1 Module-wise Implementation Details

I. Authentication Module

- Users log in using their username, password and the active academic year.
- Sessions are managed using cookies and each session carries the user role information.
- Access to pages and features is restricted based on the logged in user's role.

II. Developer Module

- Creates the Society record which is the top level entity.
- Sets up the Institute under the society.
- Creates the Super Administrator account and hands over control to the institution.

III. Super Admin Module

- Creates Administrator accounts for the institute.
- Manages user roles and their access permissions.

IV. Admin Module

- Sets up the academic year, streams, courses, subjects and divisions.
- Registers students and faculty individually or through bulk import using an Excel file.

- Creates the timetable by linking faculty, subjects, divisions and time slots.
- Promotes students from one course year to the next and reallocates them to divisions.

V. Faculty Module

- Uploads classroom photographs to mark attendance through face recognition.
- Grants substitute permission to another faculty member for a slot.
- Reviews and approves or rejects student attendance correction requests.

VI. Student Module

- Views subject wise attendance records and percentage from the personal dashboard.
- Submits correction requests if attendance seems incorrect.
- Updates profile photograph which is also used for facial enrollment.

VII. Face Recognition Module

- Receives uploaded classroom photographs from the ASP.NET application.
- Detects all faces in the photograph and extracts embeddings.
- Matches embeddings against the database and identifies students.
- Returns recognition results and the annotated photograph to the application.

4.2 Face Recognition Model Implementation

The face recognition part of the system is built as a separate Python service. It takes care of detecting faces in classroom photographs and figuring out which student each face belongs to. The whole thing runs independently and communicates with the ASP.NET web application over HTTP.

Model Used

The system uses the InsightFace library with the `buffalo_l` pretrained model. This model is built on convolutional neural networks and is good at detecting faces and generating embeddings even when lighting is not perfect or faces are slightly turned. No additional training was done on this model. The pretrained weights are used directly for both face detection and embedding extraction.

Face Detection and Embedding Extraction

When a classroom photograph is uploaded, the following steps happen:

- The image is read and converted to a NumPy array using PIL and OpenCV.

- The InsightFace model scans the image and locates all faces present in it.
- For each detected face, a 512-dimensional numerical vector called an embedding is extracted. This embedding captures the unique facial features of that person.
- These embeddings are what get compared against the stored ones during recognition.

Student Enrollment

Before a student can be recognised, they need to be enrolled in the system. During enrollment, the student uploads multiple photographs of themselves through the web portal. The system extracts an embedding from each photograph and stores all of them in the `StudentEmbeddings` table in SQL Server. Having multiple embeddings per student helps the system handle different angles and lighting conditions better during recognition.

Face Matching and Recognition

When attendance is being marked, the embeddings extracted from the classroom photograph are compared against all stored student embeddings. The comparison is done using cosine similarity, which measures how close two vectors are to each other in direction. The formula used is:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} \quad (4.1)$$

A score closer to 1 means the two faces are very similar. A threshold of **0.35** is used to decide whether a match is valid or not. If the best matching score for a detected face is below 0.35, that face is treated as unrecognised.

Duplicate Face Filtering

When multiple photographs of the same session are uploaded, the same student could appear in more than one image. To avoid counting that student twice, the system checks each detected face against all previously processed embeddings in that session. If the similarity score between two faces crosses **0.70**, the second one is treated as a duplicate and skipped.

Recognition Workflow

1. Faculty uploads one or more classroom photographs through the web interface.
2. The ASP.NET application sends the photographs to the Flask service.
3. The Flask service detects all faces and extracts their embeddings.
4. Each embedding is compared against stored student embeddings using cosine similarity.
5. Students whose similarity score meets the threshold are marked as Present.

6. Students not matched in any uploaded photograph are marked as Absent.
7. All attendance entries are saved in the `AttendanceRecords` table in SQL Server.
8. An annotated version of the photograph with bounding boxes and names is returned to the faculty dashboard.

Additional Capabilities

- Multiple photographs from the same session can be uploaded together and the system processes all of them in one request.
- Unrecognised faces are flagged separately so the faculty is aware of any unknown persons in the classroom.
- The annotated group photograph gives a quick visual confirmation of who was identified and who was not.

4.3 Development Environment and Tools

- **Visual Studio** — used for ASP.NET and C# development.
- **Visual Studio Code** — used for writing the Python Flask recognition service.
- **Git** — used for version control throughout the project.
- **SQL Server Management Studio (SSMS)** — used for database design, querying and management.
- **Postman** — used for testing the Flask API endpoints during development.

Chapter 5

TESTING

Testing is an important phase in the development of the proposed Smart Attendance System using Face Recognition. The primary objective of testing is to verify whether the system performs correctly under normal operating conditions and whether the face recognition and attendance management functionalities operate reliably throughout the academic workflow.

In this project, the proposed system was tested using an environment consisting of ASP.NET Web Forms, C#, Python Flask, InsightFace buffalo_l recognition model, Microsoft SQL Server, and Postman for API validation. Various testing scenarios were executed to evaluate user authentication, student enrollment, faculty management, face recognition processing, automatic attendance marking, database operations, and role-based access control.

The testing results demonstrate that the system successfully authenticates users across all roles, processes classroom photographs for face recognition, prevents duplicate attendance entries, and maintains accurate attendance records within the database.

5.1 Unit Testing

Unit Testing was performed to verify the functionality of each individual module in the proposed system. In this testing method, every module was tested independently to ensure that it performs its intended tasks correctly and produces the expected results.

The primary modules tested in the system include:

- Login Authentication Module
- Student Registration Module
- Faculty Registration Module
- Timetable Management Module
- Face Recognition Module
- Attendance Management Module

Through unit testing, the functionality of each module was evaluated under different operational conditions. The testing process confirmed that all modules operate correctly, respond appropriately to user inputs, and maintain reliable performance throughout the attendance management lifecycle.

5.1.1 Login Authentication Module

The Login Authentication Module was tested to verify whether the system correctly validates user credentials, assigns appropriate roles, and redirects users to their respective dashboards.

Test Case Details

TestCase ID & Name	TestCase Description	TestCase Steps	Test Data	Expected Output	Actual Output	Status
AUTH-01: Admin Login	Verify successful login with valid admin credentials.	1. Open login page. 2. Enter valid admin username and password. 3. Click Login.	Valid admin credentials.	Admin should be redirected to admin dashboard.	Admin dashboard loaded successfully.	Pass
AUTH-02: Faculty Login	Verify successful login with valid faculty credentials.	1. Open login page. 2. Enter valid faculty username and password. 3. Click Login.	Valid faculty credentials.	Faculty should be redirected to faculty dashboard.	Faculty dashboard loaded successfully.	Pass
AUTH-03: Invalid Credentials	Verify login failure for incorrect credentials.	1. Open login page. 2. Enter invalid username or password. 3. Click Login.	Invalid user credentials.	Error message should be displayed.	Login rejected with error message.	Pass
AUTH-04: Unauthorized Access Attempt	Verify protection of secured pages without login.	1. Access a secured page URL directly. 2. Submit request without authentication.	Unauthenticated request.	Access should be restricted and redirected to login.	Access denied and redirected to login page.	Fail

Table 5.1: Test Cases for Login Authentication Module

Result

The Login Authentication Module successfully validated credentials for all user roles, assigned role-based access, and redirected users to their respective dashboards. Invalid credential attempts were correctly rejected with appropriate error messages.

5.1.2 Student Registration Module

The Student Registration Module was tested to verify whether student profiles, photographs, and face embeddings are correctly stored and managed within the system.

Test Case Details

TestCase ID & Name	TestCase Description	TestCase Steps	Test Data	Expected Output	Actual Output	Status
STU-01: Student Profile Creation	Verify successful creation of a new student profile.	1. Open student registration page. 2. Enter student details. 3. Submit the form.	Valid student details.	Student profile should be created and stored in database.	Student profile created and stored successfully.	Pass
STU-02: Face Photograph Upload	Verify successful upload of student face photographs.	1. Open student profile page. 2. Upload multiple student photographs. 3. Submit for processing.	Student face photographs.	Photographs should be uploaded and embeddings generated.	Photographs uploaded and face embeddings generated successfully.	Pass
STU-03: Duplicate Student Prevention	Verify prevention of duplicate student registrations.	1. Attempt to register an existing student ID. 2. Submit the registration form.	Existing student credentials.	Duplicate entry should be rejected.	Duplicate registration rejected with error message.	Pass
STU-04: Invalid Photograph Upload	Verify system behavior when a non-face image is uploaded.	1. Upload a photograph without a visible face. 2. Submit for processing.	Non-face image file.	System should report no face detected.	No face detected error reported correctly.	Fail

Table 5.2: Test Cases for Student Registration Module

Result

The Student Registration Module successfully created student profiles, generated face embeddings from uploaded photographs, and prevented duplicate registrations. Error handling operated correctly when invalid or non-face photographs were submitted.

5.1.3 Faculty Registration Module

The Faculty Registration Module was tested to verify whether faculty profiles are correctly created, updated, and assigned to academic departments within the system.

Test Case Details

TestCase ID & Name	TestCase Description	TestCase Steps	Test Data	Expected Output	Actual Output	Status
FAC-01: Faculty Profile Creation	Verify successful creation of a new faculty profile.	1. Open faculty registration page. 2. Enter faculty details and department. 3. Submit the form.	Valid faculty details.	Faculty profile should be created successfully.	Faculty profile created and stored successfully.	Pass
FAC-02: Faculty Profile Update	Verify successful update of an existing faculty profile.	1. Open faculty profile. 2. Modify faculty details. 3. Save changes.	Updated faculty information.	Faculty profile should be updated in database.	Faculty profile updated successfully.	Pass
FAC-03: Faculty Deletion	Verify successful removal of a faculty profile.	1. Select an existing faculty record. 2. Initiate deletion. 3. Confirm deletion.	Existing faculty record.	Faculty profile should be removed from database.	Faculty profile deleted successfully.	Pass
FAC-04: Incomplete Faculty Details	Verify system behavior when mandatory fields are missing.	1. Open faculty registration page. 2. Leave mandatory fields empty. 3. Submit the form.	Incomplete faculty data.	Validation error should be displayed.	Validation error displayed and submission blocked.	Fail

Table 5.3: Test Cases for Faculty Registration Module

Result

The Faculty Registration Module successfully created, updated, and removed faculty profiles within the system. Validation mechanisms correctly prevented the submission of incomplete faculty records.

5.1.4 Face Recognition Module

The Face Recognition Module was tested to verify whether the Flask-based recognition service correctly detects multiple faces in classroom photographs, matches them against stored embeddings, and returns accurate recognition results.

Test Case Details

TestCase ID & Name	TestCase Description	TestCase Steps	Test Data	Expected Output	Actual Output	Status
FR-01: Multi-Face Detection	Verify detection of multiple faces in a single classroom photograph.	1. Upload classroom photograph with multiple students. 2. Submit to Flask recognition service.	Classroom photograph with multiple students.	All faces should be detected successfully.	All faces detected and processed successfully.	Pass
FR-02: Student Recognition	Verify correct identification of enrolled students.	1. Upload photograph of enrolled students. 2. Process through recognition service.	Photographs of enrolled students.	Students should be correctly identified.	Students identified accurately.	Pass
FR-03: Unknown Face Handling	Verify handling of unrecognized faces in the classroom photograph.	1. Upload photograph containing an unenrolled person. 2. Submit for recognition.	Photograph with unknown individual.	Unknown face should be reported separately.	Unknown face identified and reported correctly.	Pass
FR-04: Empty Image Upload	Verify system behavior when an empty or corrupt image is submitted.	1. Submit an empty or unreadable image file. 2. Process through recognition service.	Empty or corrupt image file.	Recognition service should return an appropriate error.	Error returned and attendance not recorded.	Fail

Table 5.4: Test Cases for Face Recognition Module

Result

The Face Recognition Module successfully detected multiple faces in classroom photographs, correctly identified enrolled students, and reported unrecognized faces separately. Students wearing spectacles were recognised accurately under normal indoor lighting conditions. However, the service returned an appropriate error when empty or corrupt image files were submitted.

5.1.5 Attendance Management Module

The Attendance Management Module was tested to verify whether attendance records are correctly stored, retrieved, and reported, and whether duplicate entries are prevented when multiple photographs from the same session are uploaded.

Test Case Details

TestCase ID & Name	TestCase Description	TestCase Steps	Test Data	Expected Output	Actual Output	Status
ATT-01: Automatic Attendance Marking	Verify that attendance is automatically marked after successful recognition.	1. Upload classroom photograph. 2. Process through recognition. 3. Verify attendance records.	Classroom photograph with enrolled students.	Attendance should be recorded automatically.	Attendance recorded automatically for all recognised students.	Pass
ATT-02: Duplicate Entry Prevention	Verify that duplicate attendance entries are prevented for the same session.	1. Upload a second photograph from the same session. 2. Submit for recognition and marking.	Second photograph from same class session.	Duplicate attendance entry should be prevented.	Duplicate entry detected and prevented successfully.	Pass
ATT-03: Attendance Report Generation	Verify generation of student attendance reports.	1. Open attendance reports section. 2. Select student and date range. 3. Generate report.	Student ID and date range.	Attendance report should be generated correctly.	Attendance report generated and displayed correctly.	Pass
ATT-04: Database Connection Failure	Verify system behavior when database is unavailable during attendance saving.	1. Disconnect database service. 2. Attempt to save attendance records.	Attendance data with unavailable database.	Attendance saving should fail with an error message.	Error reported and attendance not saved.	Fail

Table 5.5: Test Cases for Attendance Management Module

Result

The Attendance Management Module successfully recorded attendance automatically following face recognition, prevented duplicate entries within the same session, and generated accurate attendance reports for the specified date ranges. The system responded appropriately with an error when database connectivity was unavailable during the saving operation.

5.2 Integration Testing

Integration Testing was performed to verify whether all modules of the proposed Smart Attendance System operate correctly together within the complete deployment environment.

The primary objective of integration testing was to ensure seamless communication and coordination among the ASP.NET Web Forms frontend, C# backend, Python Flask recognition service, InsightFace model, and Microsoft SQL Server database.

During integration testing, the complete attendance workflow was evaluated end-to-end. The ASP.NET application successfully communicated with SQL Server for all data operations. The Flask API correctly received classroom photographs from the ASP.NET application, processed them through the InsightFace recognition model, and returned recognition results for attendance marking.

The interaction between the ASP.NET application and the Flask recognition service was verified through successful photograph uploads, recognition responses, and corresponding attendance records stored in the database. Role-based access control was tested across all user roles including Administrator, Faculty, and Student to confirm that each role accessed only the permitted dashboards and features.

Additional failure scenarios were tested by intentionally disconnecting the database service and submitting invalid image files. The system responded appropriately by detecting failures and preventing incorrect or incomplete attendance records from being stored.

5.3 Acceptance Testing

Acceptance Testing was conducted to verify whether the proposed system satisfies project objectives, functional requirements, and operational expectations for academic attendance management.

Testing was carried out for:

- User Authentication and Role-Based Access Control
- Student and Faculty Management Operations
- Face Recognition Accuracy under Classroom Conditions
- Automatic Attendance Recording
- Duplicate Attendance Prevention
- Attendance Report Generation
- System Integration between ASP.NET, Flask, and SQL Server

The testing process confirmed that the proposed Smart Attendance System is suitable for automated face recognition-based attendance management in an academic environment.

Chapter 6

Results and Discussions

6.1 Achieved Functionalities

Complete Web Platform: A fully functional multi-role web application has been built covering all five user roles — Developer, Super Admin, Admin, Faculty and Student. Each role has its own dashboard with features specific to that role working correctly.

Role-Based Authentication: The login system authenticates users from four separate database tables — Usermaster, SuperAdmin, FacultyMaster and StudentRegister — and redirects each user to their correct dashboard after login. Passwords are stored using SHA-256 hashing.

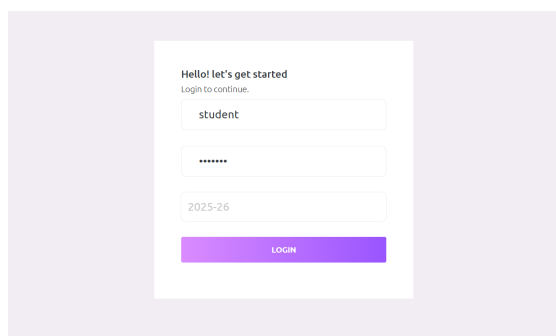


Figure 6.1: Student Login

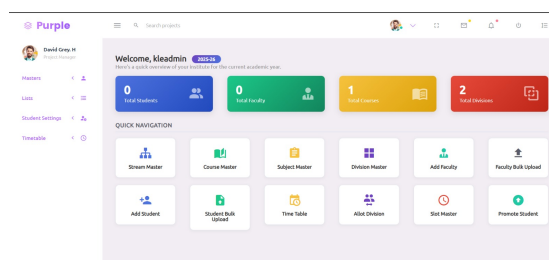


Figure 6.2: Redirect to Dashboard

Academic Management System: All academic master data modules are working — society, institute, academic year, stream, course, subject, division, timetable, faculty and student management. Bulk student import through Excel and student promotion across academic years are also functional.

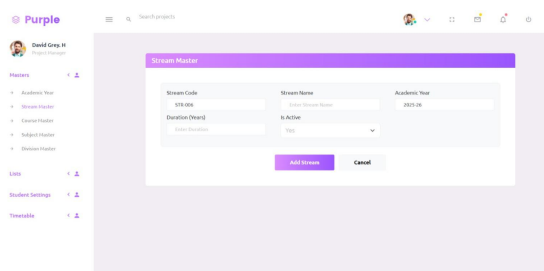


Figure 6.3: Stream Master Page

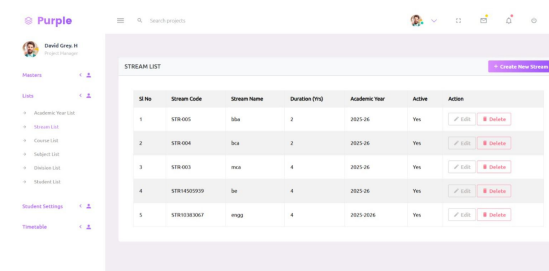
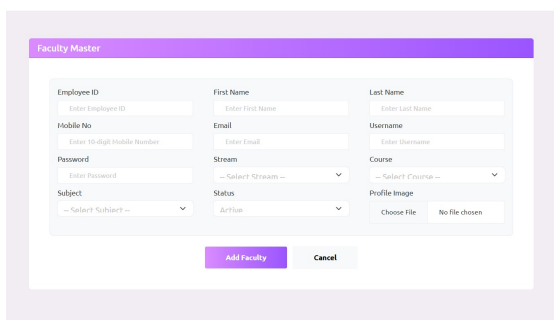
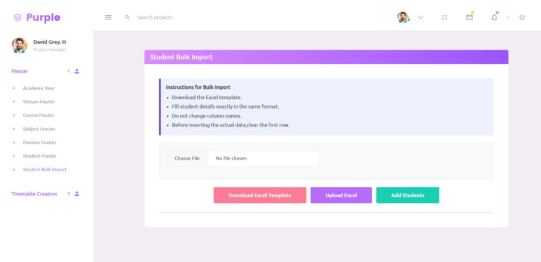


Figure 6.4: Stream List Page



The Faculty Master Page is a form for adding new faculty members. It includes fields for Employee ID, First Name, Last Name, Mobile No, Email, Username, Password, Stream, Course, Status, and Profile Image. There are also buttons for 'Add Faculty' and 'Cancel'.

Figure 6.5: Faculty Master Page



The Student Bulk Import page provides instructions for bulk import and buttons for 'Download Excel Template', 'Upload Excel', and 'Add Students'.

Figure 6.6: Student Bulk Import

Face Recognition Module: The Python based Flask service uses the InsightFace buffalo_1 pretrained model to detect faces in uploaded classroom photographs and match them against stored student embeddings. The service has been tested on a classroom dataset of 30 students and correctly handles variations in face orientation and the presence of spectacles.[8][9]



Figure 6.7: Initial Face Recognition Result

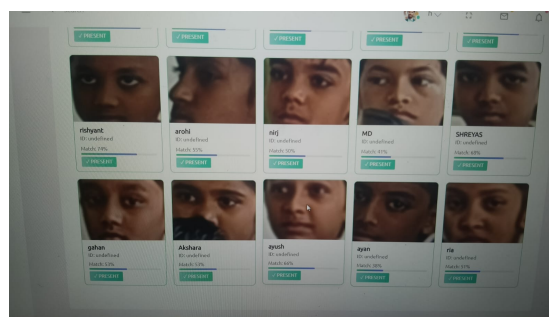


Figure 6.8: Attendance Report

Student Enrollment Portal: A dedicated portal has been built where students upload multiple photographs of themselves from different angles and lighting conditions. The system extracts and stores facial embeddings from each photograph in the StudentEmbeddings table. Real training data was collected from an educational institution to test the portal at scale.[10]

Attendance Recording Module: Once students are recognised from an uploaded photograph, their attendance is automatically recorded as Present without any manual input from the faculty. Students not matched in any photograph are marked Absent. The complete attendance log is visible on the faculty dashboard along with the annotated classroom photograph showing bounding boxes around each identified face.

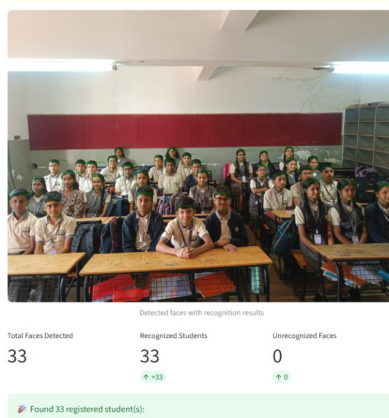


Figure 6.9: Face Recognition Accuracy

6.2 Performance Observations

- The web application loads and responds correctly across all role dashboards without any noticeable delay.
- The Flask recognition service processes classroom photographs and returns results within a few seconds for a batch of 30 students.
- The database schema maintains referential integrity through foreign key constraints across all fifteen tables.
- The responsive layout works correctly on desktop, tablet and mobile browsers without any layout issues.
- The duplicate face suppression mechanism correctly avoids counting the same student more than once when multiple photographs are uploaded for the same session.

6.3 Challenges Faced

- **3-Tier Architecture:** Setting up clean separation between the business logic, data access and presentation layers took considerable effort and iteration before the structure became stable and easy to work with.
- **Session Management:** Managing role based session data across four different user types — Developer, Super Admin, Faculty and Student — each authenticated from a different database table, required careful design of the AuthManager component.
- **Flask and ASP.NET Integration:** Getting the ASP.NET web application to correctly send photographs to the Flask service and handle the recognition response involved debugging HTTP communication, image encoding and response parsing.

- **Scaling Face Recognition:** The current setup works well for 30 students. Handling 200 or more students in a single photograph will need work on optimising the detection pipeline and possibly moving to GPU based processing.
- **Dataset Collection:** Getting five good quality photographs per student covering different angles and lighting conditions required building a structured and easy to use upload portal so that the process could be done at scale without confusion.

6.4 Current Status

- **Completed:** Full multi-role web application, role based authentication, all academic management modules, Flask based face recognition service, student enrollment portal, attendance dashboard, attendance report generation and mobile application for faculty.
- **In Progress:** Scaling the recognition pipeline to handle larger classroom batches of 200 or more students.
- **Upcoming:** Integration with live classroom cameras, GPU based processing for faster recognition, liveness detection and cloud deployment.

6.5 Discussion of Outcomes

The results from testing confirm that the system works as intended and solves the core problems it was built to address.

- **Attendance is Fully Automated:** Faculty only need to upload a classroom photograph. Everything after that — face detection, student identification, marking present or absent and saving the record — happens automatically without any manual input.
- **Proxy Attendance is Eliminated:** Since identification is based on facial features and not on a student marking themselves, there is no way for one student to mark attendance on behalf of another.
- **Records are Accurate and Centralised:** All attendance data is stored in SQL Server with subject, division, date and time information. Faculty and students can both access their respective views from the dashboard at any time.
- **System is Ready for Larger Deployment:** The modular design — with the Flask recognition service sitting separately from the ASP.NET application — means either part can be upgraded independently. Scaling to more students or switching to a faster model does not require rebuilding the web application.

Comparison with Traditional Systems:

- No time is wasted on roll calls during lectures.
- Human errors and proxy entries are removed from the process.
- All attendance data is stored centrally and is accessible instantly from any device.
- The system can be extended to more classrooms and institutions without changing the core architecture.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

This project set out to solve a very common problem in educational institutions where taking attendance manually wastes lecture time and leaves room for proxy entries. The system that has been built addresses this by using face recognition to automatically identify students from classroom photographs and mark their attendance without any manual input.

The following work was completed as part of this project:

1. **Automated Attendance Using Face Recognition:** The system detects all faces in an uploaded classroom photograph and matches them against stored student embeddings to mark attendance automatically. This removes the need for manual roll calls and eliminates the possibility of proxy attendance.
2. **Pretrained Model Integration:** The InsightFace buffalo_l pretrained model is used for face detection and 512-dimensional embedding extraction. The system was tested on a classroom dataset and produced reliable recognition results without requiring any additional model training.
3. **Student Facial Enrollment Portal:** A dedicated web portal was built where students can upload multiple photographs of themselves. The system extracts and stores embeddings from each photograph so that recognition works well across different angles and lighting conditions.
4. **Complete Web Application:** A full multi-role web application was developed using ASP.NET Web Forms with separate dashboards for Developer, Super Administrator, Administrator, Faculty and Student. Each role has access only to the features relevant to it.
5. **Flask Based Recognition Service:** The face recognition logic is implemented as a standalone Python Flask service that communicates with the ASP.NET application over HTTP. This keeps the AI part separate from the web application making it easier to maintain and update.
6. **Attendance Dashboard:** An attendance dashboard is available for both faculty and students. Faculty can view class wise attendance records, download reports and see the annotated classroom photograph showing who was recognised. Students can track their own subject wise attendance percentage.

7. **Mobile Application for Faculty:** A mobile application was developed for faculty members that allows them to upload classroom photographs, view attendance records, manage substitute requests and receive updates directly from their phone without needing to open the web application.
8. **Supporting Features:** The system also supports bulk student import, timetable management, substitute faculty assignment, student attendance correction requests and multi-institution setup through the Society and Institute structure.

Overall the system shows that face recognition can be practically used in an educational setting to make attendance management faster, more accurate and completely free of manual effort. The modular design of the system means it can be improved and scaled further without rebuilding from scratch.

7.2 Future Scope

The current system covers the complete attendance workflow from enrollment to reporting and works well for small to medium classroom sizes. However there are several areas where it can be taken further.

1. **Real-Time CCTV Camera Integration:** At present the system works on photographs uploaded by faculty. The next step would be to connect it directly with classroom CCTV cameras so that attendance is marked automatically as students enter the room without anyone needing to upload anything.
2. **Scaling to Larger Batches:** The system has been tested on a batch of 30 students. Scaling it to handle 200 or more students simultaneously will need work on optimising the recognition pipeline and possibly using better hardware or GPU based processing.
3. **Liveness Detection and Anti-Spoofing:** Currently the system cannot tell the difference between a real person and a photograph of that person held in front of the camera. Adding liveness detection would prevent such attempts and make the system more secure.
4. **Cloud Deployment:** Hosting the system on a cloud platform would make it accessible from anywhere, remove the dependency on local servers and make it easier to scale storage and processing as the number of institutions and students grows.
5. **Smart Campus Integration:** The system can be extended to connect with other campus systems such as library access, examination hall monitoring and hostel entry tracking to build a more complete smart campus solution.

References

- [1] M. Turk and A. Pentland, "Eigenfaces for recognition," *Journal of Cognitive Neuroscience*, vol. 3, no. 1, pp. 71–86, 1991.
- [2] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [4] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multi-task cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.
- [5] A. Kumar, R. Singh, and S. Patel, "Automated attendance system using face recognition," *International Journal of Engineering Research and Technology (IJERT)*, vol. 11, no. 6, pp. 812–816, 2022.
- [6] P. Patil and S. Kulkarni, "Smart attendance system using cnn-based face recognition," *International Journal of Computer Applications*, vol. 174, no. 15, pp. 20–25, 2021.
- [7] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2001, pp. 511–518. [Online]. Available: <https://ieeexplore.ieee.org/document/990517>
- [8] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "Deepface: Closing the gap to human-level performance in face verification," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1701–1708, 2014. [Online]. Available: <https://research.facebook.com/publications/deepface-closing-the-gap-to-human-level-performance-in-face-verification/>
- [9] O. Parkhi, A. Vedaldi, and A. Zisserman, "Deep face recognition," *British Machine Vision Conference (BMVC)*, 2015. [Online]. Available: <https://www.robots.ox.ac.uk/~vgg/publications/2015/Parkhi15/>
- [10] A. Liu, J. Wan, S. Escalera, and Z. Tan, "Face anti-spoofing: A survey," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1396–1415, 2018.